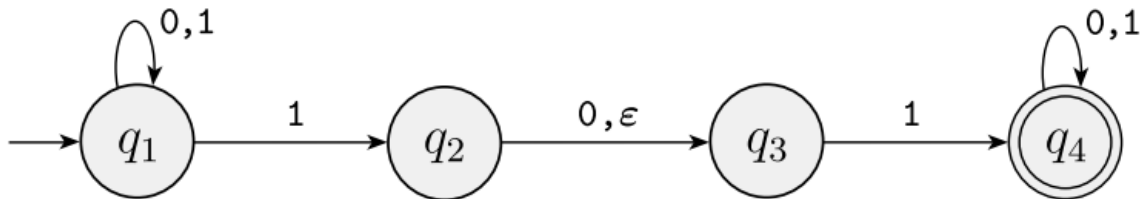


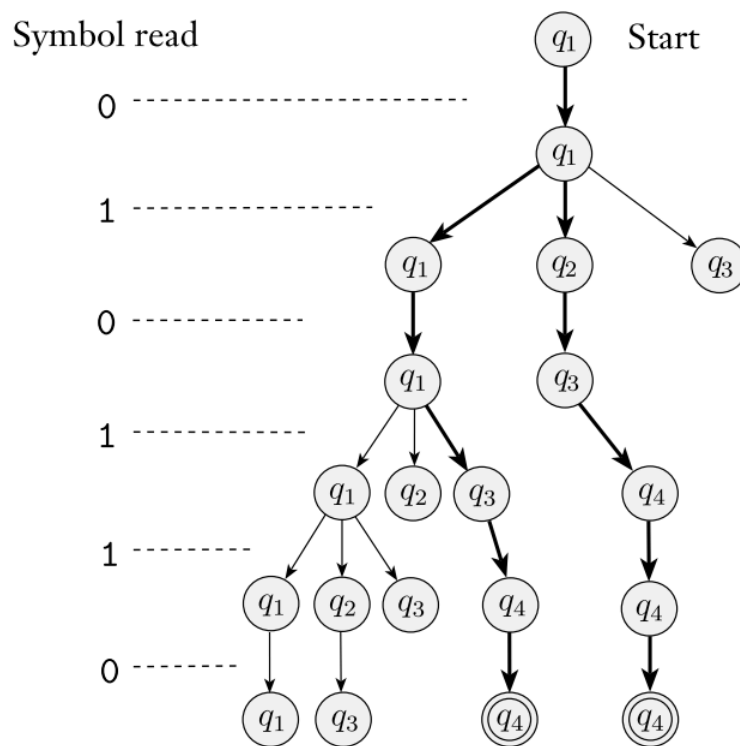
Lecture 5

How Does An NFA Machine Accept Strings?

Consider the following NFA example,



Now, consider a string 010110 as an input for the above NFA. The NFA will read every symbol of the character and concurrently stay in multiple states. After reading all the characters, if it stays in at least one accept state of the NFA, the string will be accepted.



Why Do We Need NFA?

According to NFA's properties, NFA's are just DFA's with less restrictions and every DFA is an NFA but every NFA is not DFA. DFA is a subset of NFA. So, can't we just construct

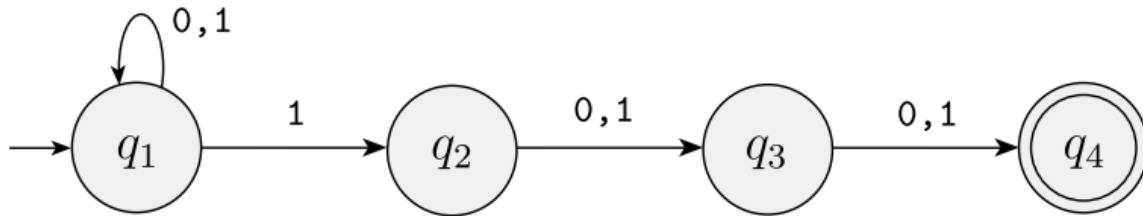
Lecture 5

DFA's when we are asked to construct NFA's? Why bother learning NFA?

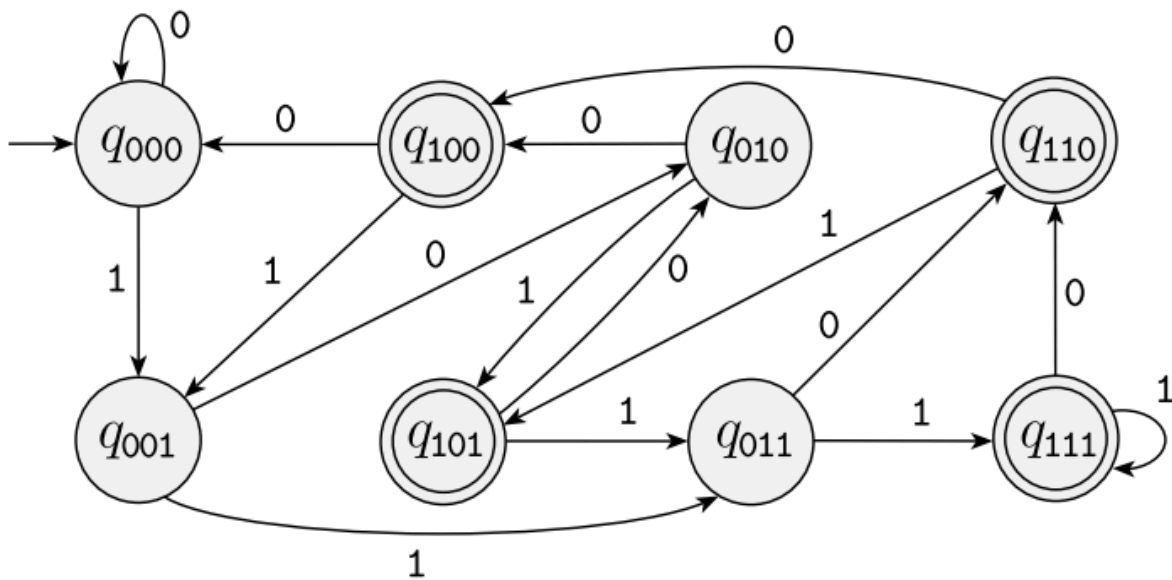
Consider the following regular language,

$L = \{w \in \{0, 1\}^* \mid w \text{ contains a 1 in the third position from the end}\}$

NFA solution for that regular language should be,



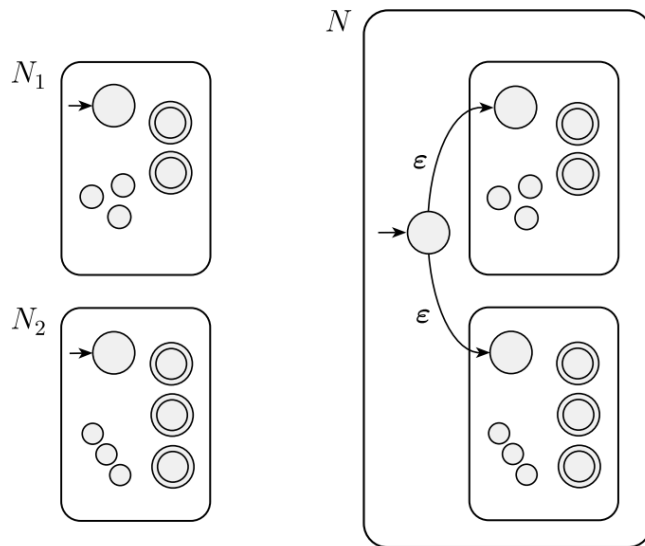
But the DFA solution for that regular language is much more complicated and tough to construct,



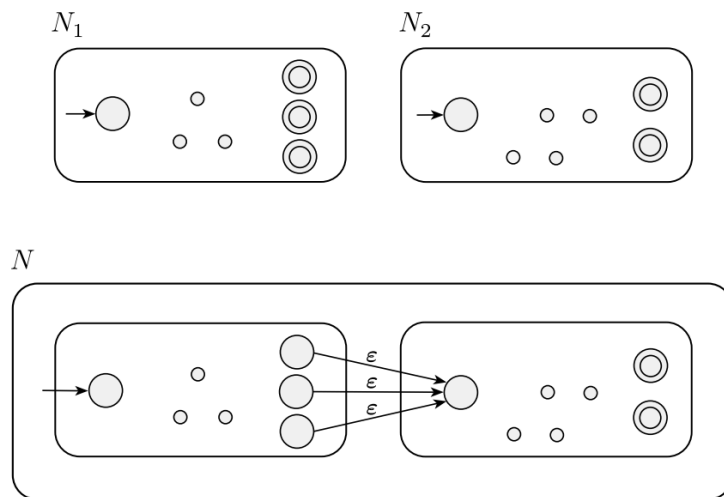
Also, NFA is easy to construct for the following Regular Operations

Lecture 5

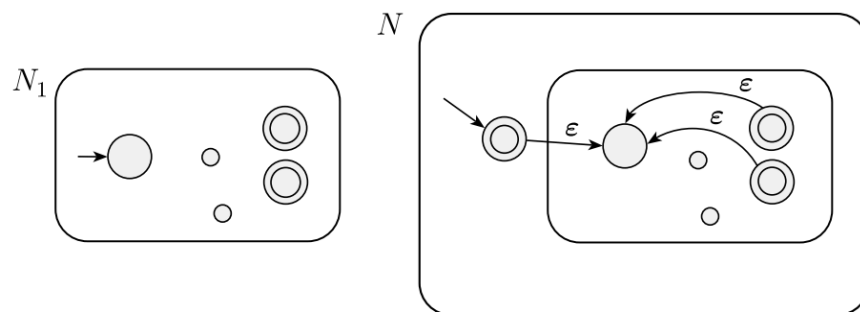
- OR/Union (U)



- Concatanation ($\circ$)



- Kleene Closure (*)



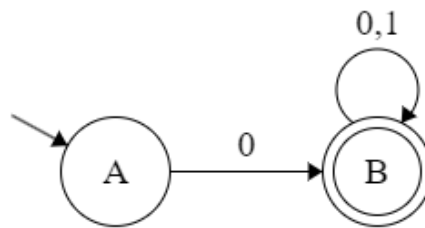
Lecture 5

But doesn't NFA concurrently compute the strings, which is costly in real life?

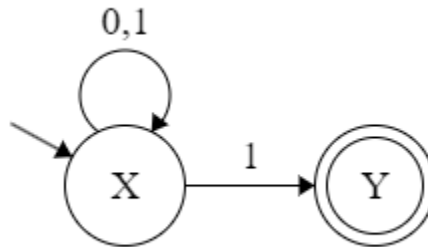
Well, we can always convert an NFA to DFA which is very easy. We'll learn NFA to DFA conversion in next lecture

NFA Examples

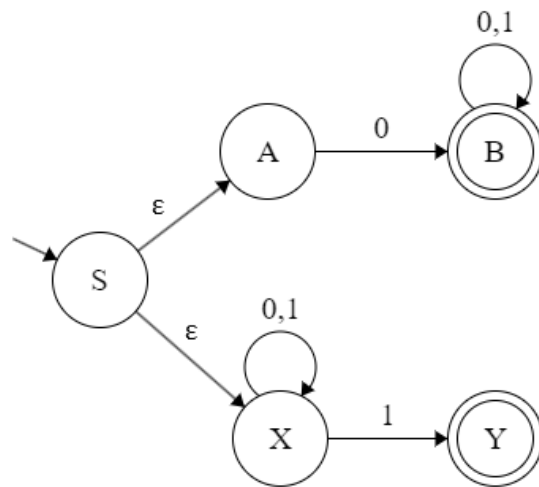
$L_1 = \{w \in \{0, 1\}^* \mid w \text{ starts with } 0\}$



$L_2 = \{w \in \{0, 1\}^* \mid w \text{ ends with } 1\}$



Lecture 5



$$L_3 = L_1 \cup L_2$$

Substring vs Subsequence

Substring is a continuous part or subpart of a string.

Example: the string "ana" is a substring (and subsequence) of banana at two different offsets:

```
banana
| | | |
ana | |
   | |
   ana
```



On the other hand, subsequence is the part of a string or sequence, that might be continuous or not but the order of the elements is maintained.

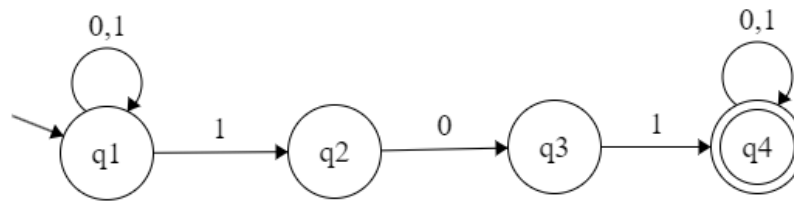
Example: the string "anna" is a subsequence of the string "banana":

```
banana
| |  | |
an na
```

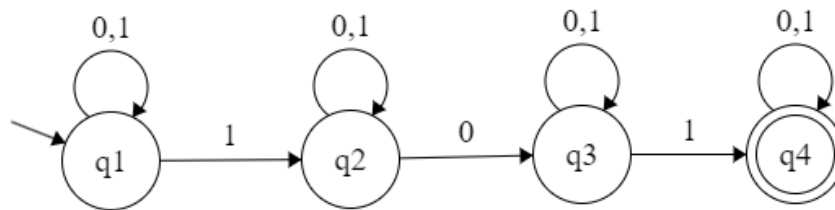


Lecture 5

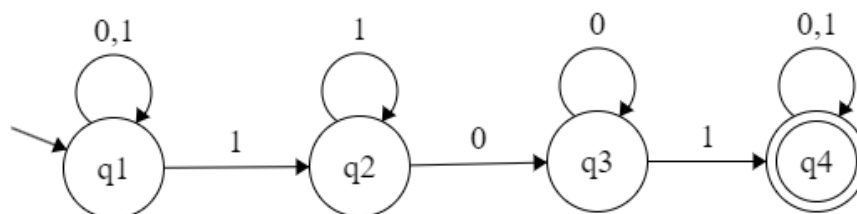
$L_4 = \{w \in \{0, 1\}^* \mid w \text{ has substring } 101\}$



$L_5 = \{w \in \{0, 1\}^* \mid w \text{ has subsequence } 101\}$

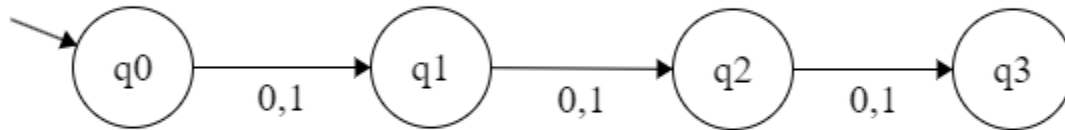


Another solution could be,

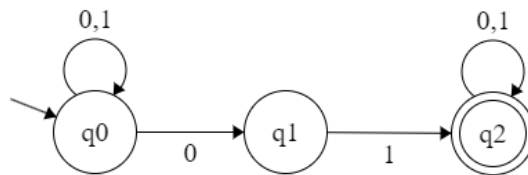


Lecture 5

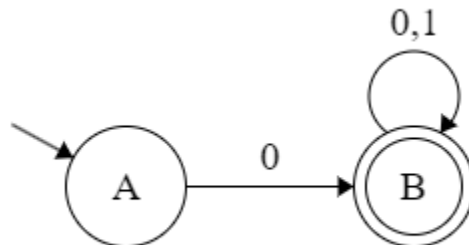
$L_6 = \{w \in \{0, 1\}^* \mid \text{length of } w \text{ is } 3\}$



$L_7 = \{w \in \{0, 1\}^* \mid w \text{ has substring } 01\}$

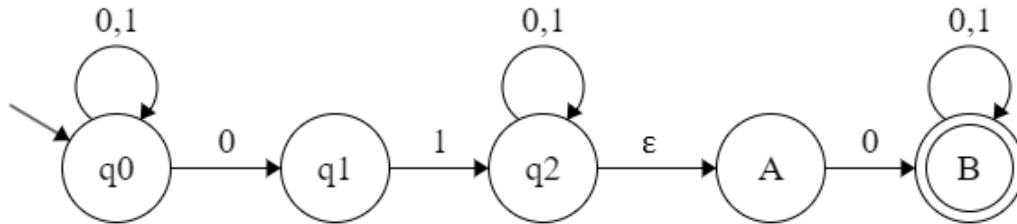


$L_8 = \{w \in \{0, 1\}^* \mid w \text{ starts with } 0\}$ [same as L_1 but showing it for the example L_9]

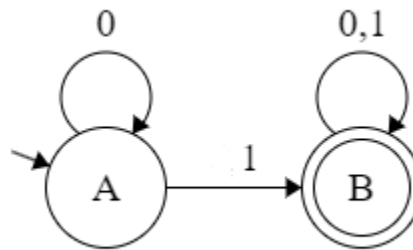


Lecture 5

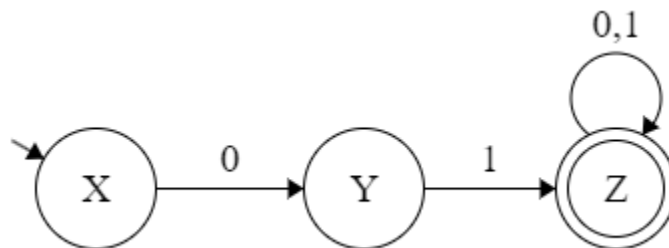
$$L_9 = L_7 \cdot L_8$$



$$L_{10} = \{w \in \{0, 1\}^* \mid w \text{ has } 1\}$$

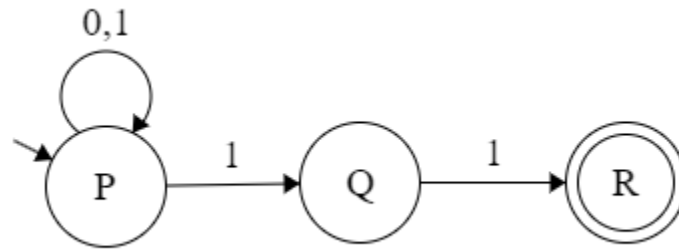


$$L_{11} = \{w \in \{0, 1\}^* \mid w \text{ starts with } 01\}$$

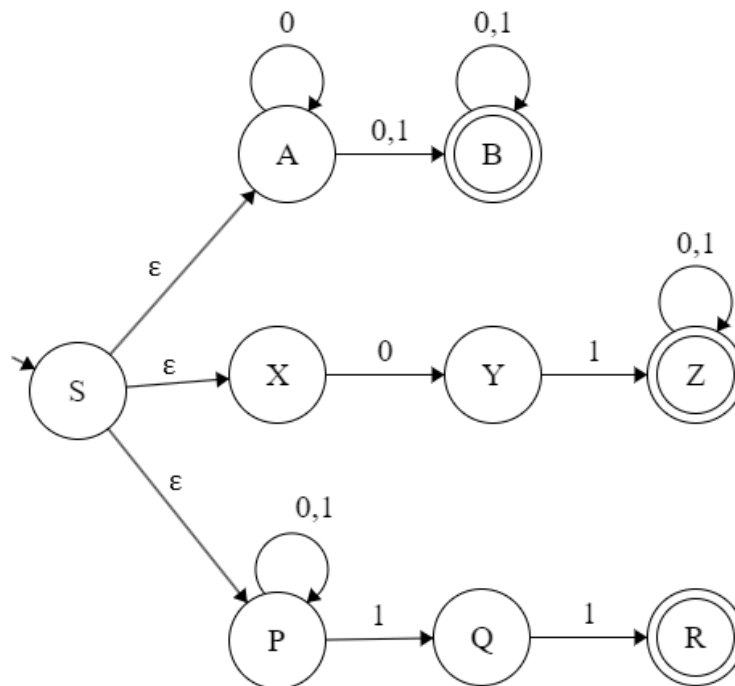


Lecture 5

$L_{12} = \{w \in \{0, 1\}^* \mid w \text{ ends with } 11\}$

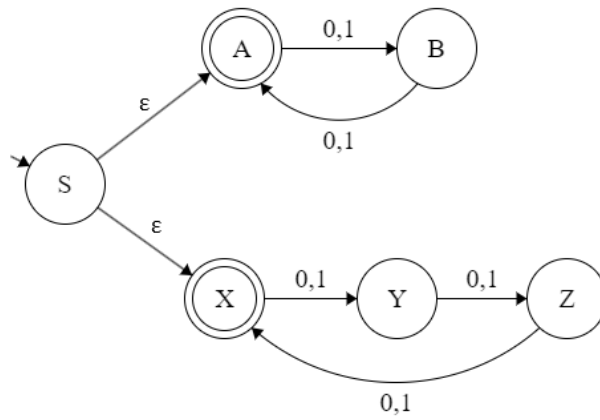


$L_{13} = L_{10} \cup L_{11} \cup L_{12}$ OR $L_{13} = L_{10} \mid L_{11} \mid L_{12}$

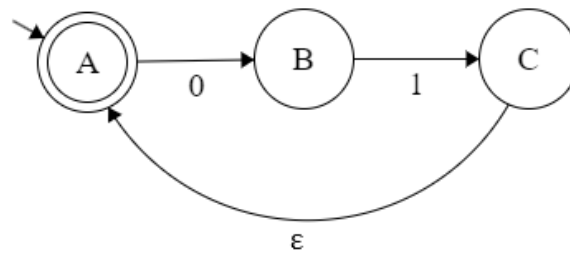


Lecture 5

$L_{14} = \{w \in \{0, 1\}^* \mid \text{length of } w \text{ is multiple of 2 or 3}\}$



$L_{15} = \{w \in \{0, 1\}^* \mid w \text{ is } (01)^m, m \geq 0\}$



$L_{16} = \{w \in \{0, 1\}^* \mid w \text{ is } (01)^m, m \geq 1\}$

Lecture 5

